
RAPPORT DES TP_s

RÉALITÉ AUGMENTÉE FROM SCRATCH

Stabilité numérique, algèbre linéaire et pipeline graphique

UE	Coder pour la VR, l'XR et l'AR 2
Étudiant	DIFFO KENNE GARNEL
Matricule	24P816
Établissement	ENSPY
Date	03 Avril 2026

0 Contents

1	Introduction	3
2	Précision Numérique	4
2.1	Représentation IEEE 754 des flottants	4
2.1.1	Pourquoi 0.1f n'est pas représentable exactement	4
2.1.2	Valeurs spéciales IEEE 754	4
2.2	Epsilon machine et ULP	4
2.3	Erreurs d'arrondi et algorithmes stables	4
2.3.1	Cancellation catastrophique	4
2.3.2	Sommation de Kahan	5
2.3.3	Variance : méthode naïve vs Welford	5
2.4	Résultats expérimentaux	5
3	Vecteurs et Algèbre Linéaire	6
3.1	Vec2d : Vecteurs 2D	6
3.1.1	Produit scalaire	6
3.1.2	Produit vectoriel 2D (scalaire)	6
3.1.3	Normalisation	6
3.2	Vec3d : Vecteurs 3D	6
3.2.1	Produit vectoriel 3D	6
3.2.2	Orthogonalisation de Gram-Schmidt	6
3.2.3	Rejet orthogonal	6
3.3	Vec4d : Coordonnées homogènes	6
3.3.1	Projection perspective simple	6
4	Matrices et Transformations	7
4.1	Matrices 4×4 homogènes	7
4.1.1	Composition TRS	7
4.1.2	Décomposition TRS	7
4.1.3	Inversion numérique de matrice	7
4.1.4	Caméra LookAt	7
4.1.5	Matrice de projection perspective	7
5	Rasterisation Logicielle	8
5.1	Pipeline graphique	8
5.2	Composants implémentés	8
5.2.1	Cube unitaire	8
5.2.2	Algorithme de tracé de lignes	8
5.2.3	Format de sortie PPM	8
5.2.4	Animation par rotation	8
6	Quaternions et Animation	9
6.1	Représentation des quaternions	9
6.1.1	Rotation d'un vecteur	9
6.1.2	Conversion quaternion $\rightarrow$ matrice 3×3	9
6.1.3	Inverse et identité	9
6.2	Interpolation sphérique (SLERP)	9

6.2.1	LERP vs SLERP	9
6.2.2	Formule SLERP	9
6.2.3	Animation	9
7	Tests et Validation	10
8	Conclusion	11

1 Introduction

Ce rapport présente l'ensemble des travaux pratiques réalisés dans le cadre du cours *Réalité Augmentée from Scratch*, couvrant les quatre premières semaines du module **NkMath**. L'objectif principal est de construire une bibliothèque mathématique complète en C++17 from scratch, sans dépendances tierces, en se concentrant sur les thèmes suivants :

- **Semaine 1** — Stabilité numérique et représentation IEEE 754 des flottants
- **Semaine 2** — Structures vectorielles (Vec2d, Vec3d, Vec4d) et algèbre linéaire de base
- **Semaine 3** — Matrices 4×4 , transformations affines et inversion numérique
- **Semaine 4** — Quaternions, interpolation sphérique (SLERP) et animation

Chaque section détaille les concepts théoriques, les implémentations pratiques et les tests unitaires correspondants. L'ensemble est couvert par plus de **40 tests** validant la robustesse numérique et les cas limites.

2 Précision Numérique

2.1 Représentation IEEE 754 des flottants

Un nombre flottant IEEE 754 simple précision (32 bits) est structuré comme suit :

- **Bit 31** : Signe (0 = positif, 1 = négatif)
- **Bits 30–23** : Exposant biaisé (8 bits, biais = 127)
- **Bits 22–0** : Mantisse (23 bits, avec bit implicite 1)

La valeur représentée est :

$$(-1)^s \times 2^{(e-127)} \times (1 + m \times 2^{-23}) \quad (1)$$

où s est le bit de signe, e l'exposant biaisé, et m la mantisse.

2.1.1 Pourquoi 0.1f n'est pas représentable exactement

$0.1_{10} = 0.00011002$ en binaire. La mantisse de 23 bits ne peut représenter que les 23 premiers bits après la virgule, conduisant à l'approximation :

$$0.1 \approx 0.100000001490116119384765625$$

Cette erreur d'arrondi est la source de nombreux problèmes numériques dans les calculs AR.

2.1.2 Valeurs spéciales IEEE 754

Valeur	Exposant	Mantisse	Remarque
± 0	0	0	Zéros signés
$\pm \infty$	255	0	Résultat d'overflow
NaN	255	$\neq 0$	Contagieux, tester avec <code>std::isnan()</code>
Subnormal	0	$\neq 0$	Très petits nombres

2.2 Epsilon machine et ULP

L'epsilon machine ε est la plus petite valeur telle que $1.0 + \varepsilon \neq 1.0$:

$$\varepsilon_{\text{float}} \approx 1.19 \times 10^{-7}, \quad \varepsilon_{\text{double}} \approx 2.22 \times 10^{-16}$$

L'ULP (*Unit in the Last Place*) mesure l'espacement entre flottants consécutifs. Il varie avec la magnitude :

- Près de 1.0 : $\text{ULP} \approx 1.19 \times 10^{-7}$
- Près de 1000.0 : $\text{ULP} \approx 1.22 \times 10^{-4}$

2.3 Erreurs d'arrondi et algorithmes stables

2.3.1 Cancellation catastrophique

La soustraction de deux nombres presque égaux perd des bits significatifs :

$$a = 1\,234\,567.89, \quad b = 1\,234\,567.00 \implies a - b = 0.89 \approx 0.875$$

2.3.2 Sommation de Kahan

L'algorithme compense les erreurs accumulées à chaque étape :

$$y = x_i - c \quad (2)$$

$$t = s + y \quad (3)$$

$$c = (t - s) - y \quad (4)$$

$$s = t \quad (5)$$

où c est le terme de compensation.

2.3.3 Variance : méthode naïve vs Welford

La méthode naïve :

$$\sigma^2 = \frac{\sum x_i^2 - \left(\sum x_i\right)^2 / n}{n - 1}$$

est instable numériquement (peut produire une variance négative).

La méthode de Welford maintient la précision en une seule passe :

$$\delta = x_i - \bar{x}_{i-1} \quad (6)$$

$$\bar{x}_i = \bar{x}_{i-1} + \delta / i \quad (7)$$

$$M2_i = M2_{i-1} + \delta (x_i - \bar{x}_i) \quad (8)$$

2.4 Résultats expérimentaux

Méthode	Résultat (10^6 additions de 0.1f)	Erreur
<code>std::accumulate</code>	100 011.55	+11.55
Kahan	100 000.00	≈ 0

Sur le jeu $\{10^8, 10^8, 1.0, 2.0\}$, la méthode naïve peut retourner une variance négative, tandis que Welford reste stable.

3 Vecteurs et Algèbre Linéaire

3.1 Vec2d : Vecteurs 2D

Implémentation avec opérateurs arithmétiques complets, accès par index et fonctions utilitaires.

3.1.1 Produit scalaire

$$\mathbf{a} \cdot \mathbf{b} = a_x b_x + a_y b_y$$

3.1.2 Produit vectoriel 2D (scalaire)

$$\mathbf{a} \times \mathbf{b} = a_x b_y - a_y b_x$$

Donne l'aire orientée du parallélogramme défini par $\mathbf{a}$ et $\mathbf{b}$.

3.1.3 Normalisation

$$\hat{\mathbf{u}} = \frac{\mathbf{u}}{\|\mathbf{u}\|}, \quad \|\mathbf{u}\| = \sqrt{u_x^2 + u_y^2}$$

3.2 Vec3d : Vecteurs 3D

3.2.1 Produit vectoriel 3D

$$\mathbf{a} \times \mathbf{b} = \begin{pmatrix} a_y b_z - a_z b_y \\ a_z b_x - a_x b_z \\ a_x b_y - a_y b_x \end{pmatrix}$$

Suit la règle de la main droite.

3.2.2 Orthogonalisation de Gram-Schmidt

Construction d'une base orthonormée $\{\mathbf{u}_1, \mathbf{u}_2, \mathbf{u}_3\}$ à partir de trois vecteurs $\mathbf{a}, \mathbf{b}, \mathbf{c}$:

$$\mathbf{u}_1 = \frac{\mathbf{a}}{\|\mathbf{a}\|} \tag{9}$$

$$\mathbf{v}_1 = \mathbf{b} - \text{proj}_{\mathbf{u}_1}(\mathbf{b}), \quad \mathbf{u}_2 = \frac{\mathbf{v}_1}{\|\mathbf{v}_1\|} \tag{10}$$

$$\mathbf{v}_2 = \mathbf{c} - \text{proj}_{\mathbf{u}_1}(\mathbf{c}) - \text{proj}_{\mathbf{u}_2}(\mathbf{c}), \quad \mathbf{u}_3 = \frac{\mathbf{v}_2}{\|\mathbf{v}_2\|} \tag{11}$$

avec $\text{proj}_{\mathbf{u}}(\mathbf{v}) = (\mathbf{v} \cdot \mathbf{u}) \mathbf{u}$.

3.2.3 Rejet orthogonal

$$\text{rej}_{\mathbf{u}}(\mathbf{v}) = \mathbf{v} - \text{proj}_{\mathbf{u}}(\mathbf{v})$$

3.3 Vec4d : Coordonnées homogènes

Utilisées pour les transformations affines ; le quatrième composant w permet d'encoder les translations dans une multiplication matricielle.

3.3.1 Projection perspective simple

Pour un point 3D (X, Y, Z) , la projection 2D est :

$$u = f \frac{X}{Z}, \quad v = f \frac{Y}{Z}$$

avec f le facteur de champ de vision.

4 Matrices et Transformations

4.1 Matrices 4×4 homogènes

Les matrices 4×4 représentent les transformations affines 3D sous la forme :

$$M = \begin{pmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^T & 1 \end{pmatrix}$$

où $\mathbf{R} \in \mathbb{R}^{3 \times 3}$ est la rotation et $\mathbf{t} \in \mathbb{R}^3$ la translation.

4.1.1 Composition TRS

Les transformations Translation–Rotation–Scale se composent dans l'ordre :

$$M = T \cdot R \cdot S$$

4.1.2 Décomposition TRS

Extraction des paramètres depuis M :

- **Translation** : colonne 4 de M
- **Scale** : normes des colonnes de la sous-matrice 3×3
- **Rotation** : colonnes normalisées de la sous-matrice 3×3

4.1.3 Inversion numérique de matrice

Utilisation de l'algorithme de Gauss-Jordan avec **pivot partiel** pour assurer la stabilité. Testé sur matrices aléatoires et cas singuliers.

4.1.4 Caméra LookAt

Construction de la matrice de vue depuis un triplet (eye, target, up) :

$$\mathbf{z} = \text{normalize}(\text{eye} - \text{target}) \quad (12)$$

$$\mathbf{x} = \text{normalize}(\text{up} \times \mathbf{z}) \quad (13)$$

$$\mathbf{y} = \mathbf{z} \times \mathbf{x} \quad (14)$$

4.1.5 Matrice de projection perspective

$$P = \begin{pmatrix} \frac{f}{\text{aspect}} & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & \frac{\text{far} + \text{near}}{\text{near} - \text{far}} & \frac{2 \cdot \text{far} \cdot \text{near}}{\text{near} - \text{far}} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

avec $f = \frac{1}{\tan(\theta/2)}$ et θ le FOV vertical.

5 Rasterisation Logicielle

5.1 Pipeline graphique

Le pipeline implémenté suit quatre étapes :

1. **Transformation modèle** : espace objet $\rightarrow$ espace monde
2. **Transformation vue** : espace monde $\rightarrow$ espace caméra
3. **Projection** : espace caméra $\rightarrow$ NDC
4. **Rasterisation** : NDC $\rightarrow$ pixels écran

La transformation complète d'un point local vers l'écran s'écrit :

$$\mathbf{p}_{\text{écran}} = P \cdot V \cdot M \cdot \mathbf{p}_{\text{local}}$$

5.2 Composants implémentés

5.2.1 Cube unitaire

Définition des 8 sommets et 12 arêtes d'un cube centré en $(0, 0, 0)$.

5.2.2 Algorithme de tracé de lignes

Utilisation de l'algorithme de **Bresenham** pour rasteriser les arêtes entre sommets projetés.

5.2.3 Format de sortie PPM

Sauvegarde au format binaire P6 :

```
P6
width height
255
<data RGB>
```

5.2.4 Animation par rotation

Rotation autour de l'axe Y :

$$R_y(\theta) = \begin{pmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Génération de **10 frames** pour simuler une rotation complète à 360.

6 Quaternions et Animation

6.1 Représentation des quaternions

Un quaternion représente une rotation 3D :

$$\mathbf{q} = w + x\mathbf{i} + y\mathbf{j} + z\mathbf{k}$$

avec la contrainte d'unitarité $w^2 + x^2 + y^2 + z^2 = 1$.

6.1.1 Rotation d'un vecteur

$$\mathbf{v}' = \mathbf{q} \mathbf{v} \mathbf{q}^{-1}$$

6.1.2 Conversion quaternion $\rightarrow$ matrice 3×3

$$R = \begin{pmatrix} 1 - 2y^2 - 2z^2 & 2xy - 2wz & 2xz + 2wy \\ 2xy + 2wz & 1 - 2x^2 - 2z^2 & 2yz - 2wx \\ 2xz - 2wy & 2yz + 2wx & 1 - 2x^2 - 2y^2 \end{pmatrix}$$

6.1.3 Inverse et identité

Pour un quaternion unitaire : $\mathbf{q}^{-1} = \bar{\mathbf{q}} = (w, -x, -y, -z)$.

Vérification : $\mathbf{q} \cdot \mathbf{q}^{-1} = 1$.

La stabilité numérique du cycle $\mathbf{q} \rightarrow R \rightarrow \mathbf{q}$ a été vérifiée sur **50 cas aléatoires**.

6.2 Interpolation sphérique (SLERP)

6.2.1 LERP vs SLERP

- **LERP** : interpolation linéaire dans l'espace des quaternions — vitesse angulaire *non constante*
- **SLERP** : interpolation sur le grand cercle de S^3 — vitesse angulaire *constante*

6.2.2 Formule SLERP

$$\text{slerp}(\mathbf{q}_1, \mathbf{q}_2, t) = \frac{\sin((1-t)\theta)}{\sin \theta} \mathbf{q}_1 + \frac{\sin(t\theta)}{\sin \theta} \mathbf{q}_2$$

avec $\cos \theta = \mathbf{q}_1 \cdot \mathbf{q}_2$.

6.2.3 Animation

Interpolation entre deux orientations sur **60 frames**, avec rendu du cube en rotation et comparaison visuelle LERP/SLERP.

7 Tests et Validation

Le fichier `TestsDesTPs.cpp` contient 11 suites de tests couvrant l'ensemble des fonctionnalités :

Suite	Thème	Description
TP1	Flottants IEEE 754	Inspection bit-à-bit (<code>inspectFloat</code>)
TP2	Précision	Somme Kahan, variance Welford, epsilon machine
TP3	Utilitaires flottants	33 tests : <code>isFiniteValid</code> , <code>nearlyZero</code> , <code>approxEq</code>
TP4	Vec2d	20 tests : dot, cross 2D, normalisation, accès
TP5	Vec3d / Gram-Schmidt	16 tests : cross 3D, ortho. sur 10 triplets aléatoires
TP6	Projection Vec4d	Rendu d'un cube avec sommets projetés
TP7	Mat4d / Inversion	10 tests : matrices aléatoires et singulières
TP8	Rasterisation	10 frames d'animation du cube en rotation
TP9	Décomposition TRS	20 tests sur matrices aléatoires
TP10	Quaternions	50 tests $\mathbf{q} \rightarrow R \rightarrow \mathbf{q}$, 50 tests d'inverse
TP11	Animation SLERP	60 frames SLERP et LERP pour comparaison

Au total, **plus de 40 assertions** couvrent les cas nominaux, les cas limites et la robustesse numérique.

8 Conclusion

Ce projet a permis de construire les fondations mathématiques essentielles d'un moteur de réalité augmentée from scratch. Les principales réalisations sont :

- **Précision numérique** — Analyse approfondie de l'IEEE 754 et implémentation des algorithmes stables Kahan et Welford
- **Algèbre vectorielle** — Structures Vec2d, Vec3d, Vec4d avec algèbre linéaire complète (dot, cross, Gram-Schmidt)
- **Matrices 4×4** — Inversion numérique par Gauss-Jordan, composition et décomposition TRS
- **Pipeline graphique** — Rasterisation logicielle avec projection perspective, algorithme de Bresenham et export PPM
- **Quaternions** — Représentation compacte des rotations avec interpolation SLERP pour une animation fluide

L'ensemble est validé par plus de 40 tests unitaires couvrant les cas limites et la robustesse numérique. La bibliothèque **NkMath** constitue une base solide pour les modules suivants : SVD, traitement d'image et calibration caméra.